Video Script: No Training, No Tuning

Narration for the 13-slide TabPFN deck

Length: 1,614 words, about 12 minutes at a relaxed pace (around 135 words per minute). Timestamps are estimates; check them against your recording.

Recording tips: Pause for a breath at each slide change. Point at the figure as you describe it on slides 4 to 6. The same text is in each slide’s speaker notes, so you can read from Presenter View while recording.

Introduction

Slide 1 · Title 0:00

Hi everyone, I'm Anshita Shah, a software engineering graduate student at San José State University. Today I want to walk you through a paper that changed how I think about tabular data: "Accurate predictions on small data with a tabular foundation model," published in Nature in January 2025 by Noah Hollmann, Samuel Müller, Frank Hutter and their colleagues at the University of Freiburg and Prior Labs. The model is called TabPFN, and the reason I titled this "No Training, No Tuning" will become clear in a minute. I'll cover the problem, the core idea, how the model was trained, its architecture, how it was evaluated, what came after, where it falls short, and my own take.

Slide 2 · Twenty years of trees 0:45

Let's start with the status quo. If you've worked with spreadsheet-style data, like customers, transactions or lab results, you know the default answer. For roughly twenty years it has been gradient-boosted decision trees: XGBoost, LightGBM and CatBoost. Deep learning took over images, audio and text, but tables were different. Rows and columns don't have the neat structure of pixels or words, datasets are often small, and trees are fast and easy to tune. A neural network might win on a good day, but trees were very hard to beat. So when a paper in Nature says a neural network now beats them, it deserves a careful look.

Slide 3 · The headline result 1:40

Here's the headline. On classification tasks with up to ten thousand rows, TabPFN outperformed an ensemble of the strongest baselines, each tuned for four hours, in just 2.8 seconds. And the surprising part isn't only the speed. TabPFN does no training on your data at all. It doesn't update a single weight. So the obvious question is: how can a model predict well on a dataset it was never trained on?

How it works

Slide 4 · Learn the learning algorithm 2:15

The answer is that the authors changed what gets trained. In the usual workflow, at the top, every dataset gets its own model. You train it, tune its hyperparameters with cross-validation, repeat, and finally predict. TabPFN flips this around. The authors trained one model, once, whose only job is to look at a labelled training set and predict the labels of new rows. When you use it, you pass in your training rows and your test rows together, and you get predictions back in one forward pass. If that sounds familiar, it's the same idea as in-context learning in large language models, where you put a few examples into the prompt. Here, the prompt is your entire training table.

Slide 5 · Worlds that never existed 3:15

To learn how to learn from tables, a model needs to see an enormous number of tables, and real ones are scarce, messy and often private. Training on public benchmarks would also contaminate the very datasets used to evaluate it. So the authors generated their own. Each synthetic dataset starts from a structural causal model: a random graph of variables, where each variable is computed from its parents with a random function, such as a linear map, a small neural network or a threshold rule, plus some noise. One variable becomes the label. They sample a table from this little world, hide some labels, and train the transformer to predict them, and they repeated this about 130 million times. This is where the name comes from. The space of all these worlds is the prior, and the network is fitted to data from that prior, so it learns to approximate Bayesian prediction. The whole pre-training took roughly two weeks on a single eight-GPU machine, which is small by foundation-model standards.

Slide 6 · A transformer built for tables 4:35

Now the architecture. A language model reads a sequence, but a table isn't a sequence. If you shuffle the rows or the columns, the answer shouldn't change. So TabPFN treats every cell as a token and stacks twelve layers, and each layer does two kinds of attention. First, across the row, shown in blue: each cell looks at the other features of the same example, which captures interactions like high usage combined with a short contract. Second, down the column, shown in orange: each cell looks at the same feature in other examples, which tells it what's normal or unusual for that feature. Test rows read from the training rows but not from each other, so one customer's prediction never depends on who else is in the test batch. Because nothing depends on order, the same network handles tables of different shapes, and its output is a full probability distribution, so you get uncertainty for free.

Evidence

Slide 7 · How they tested it 5:50

A claim this big needs a fair fight, and this was the part I found most convincing. The authors used two standard benchmark suites, the AutoML Benchmark and OpenML-CTR23: 29

classification and 28 regression datasets, with up to ten thousand rows and five hundred features. Classification was scored with ROC AUC and accuracy, and regression with R-squared and negative RMSE, normalized per dataset so that one is the best method and zero the worst. The baselines got every advantage: CatBoost, XGBoost, LightGBM and other standard models, tuned with random search and five-fold cross-validation, with budgets from thirty seconds up to four hours. TabPFN got no tuning at all. They also ran stress tests, adding useless features, injecting outliers, and dropping samples or features, which act like ablation studies on the data.

Slide 8 · What they found 7:00

So what did they find? First, accuracy and speed: the result I showed earlier, 2.8 seconds against four hours, and more broadly, TabPFN outperformed previous methods by a wide margin on datasets of up to ten thousand samples. Second, data efficiency: according to the University of Freiburg, TabPFN needs only about half as much data to reach the accuracy of the previous best model. If you only have a few hundred labelled examples, that makes a real difference. Third, robustness: under the stress tests TabPFN held up well, and it's especially strong on small tables with many outliers or many missing values.

Slide 9 · More than a classifier 7:50

What makes this especially interesting for data mining is that TabPFN models the whole table, not just one label column. So the same network supports density estimation, which you can use to spot outliers and anomalies. It can generate synthetic rows with the same statistical structure as the original. It can fill in missing values by predicting any column from the others. It produces reusable embeddings for each row, which you could feed into clustering. And it can be fine-tuned on a family of related datasets. That's a lot of classic data mining tasks from one model.

Slide 10 · From 1,000 rows to a million 8:35

The Nature paper is actually version two. The first TabPFN appeared in 2022 and was presented at ICLR 2023, and it handled only up to a thousand rows and a hundred numeric features. Version two raised that to ten thousand rows and five hundred features. Then TabPFN-2.5, in November 2025, went to fifty thousand rows and two thousand features, about twenty times more cells. Its report claims it leads the TabArena benchmark, matches AutoGluon 1.4, a tuned four-hour ensemble, and beats default XGBoost on every small-to-medium classification dataset tested. It also added distillation, which compresses TabPFN into a small MLP or tree ensemble for fast deployment. Later versions released in 2026 are reported to reach around a million rows.

Reflection

Slide 11 · Where it falls short 9:40

Now, where does it fall short? Size: version two is built for up to ten thousand rows, and large datasets are still home ground for boosted trees. Prediction cost: there's no training step, but the work doesn't disappear. Each prediction has to look at the training rows, so inference is heavier than calling a trained tree model, especially without a GPU, and that's largely why distillation

was added. Interpretability: with XGBoost you can inspect trees and feature importances, while TabPFN is a large network, and explaining its predictions is still open research. Prior mismatch: the model is only as good as the worlds its generator can imagine, and a 2025 study reported weaker performance on very wide, high-dimensional data. And licensing: the code is open source, but the newer model weights are non-commercial, so check before you put it in a product.

Slide 12 · My two cents 10:50

Here's my own take. When I built a customer-churn model with XGBoost on a telecom dataset, most of my time didn't go into choosing the model. It went into tuning it: learning rate, tree depth, number of trees, and cross-validating again and again. A churn table with a few thousand customers sits right inside TabPFN's comfort zone, so my next step is to run it as a baseline next to my tuned XGBoost and see whether those hours of tuning bought anything. What I find most interesting from a data mining point of view is where the knowledge lives. Normally it shows up in feature engineering and hyperparameter search. Here it has moved into the data generator: choosing which causal worlds to simulate is deciding what the model believes before it sees a single real row. But I wouldn't skip the fundamentals. Fast predictions don't protect you from data leakage, a badly chosen metric or class imbalance. TabPFN removes the tuning loop, not the thinking.

Slide 13 · Key takeaways 12:05

To wrap up. TabPFN is a transformer pre-trained on about 130 million synthetic tables that predicts on a new dataset in one forward pass, with no training or tuning. Its two-way attention, across rows and down columns, respects how tables actually work. On datasets of up to ten thousand rows, it beat four-hour tuned baselines in under three seconds and held up well against noise and outliers. And its limits, which are scale, inference cost, interpretability and licensing, are exactly what newer versions are working on. I wouldn't call trees dead, but for small tables they're no longer the automatic default. You can read the full article on my Medium; the link is on this slide. Thanks for watching.